

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: K. Sirisha

Roll No.: A24126552260

Section: B

EXPERIMENT 3 - EXPRESS.JS STUDENT SERVER (CO-2)

1. TITLE

Develop a Basic Express.js Student Server with Multiple Routes Using Node.js, Express Routing, HTTP Methods, and Middleware.

2. AIM

To design and implement a basic Express.js web server that defines the routes `/`, `/students`, and `/about`, where the `/students` route returns a list of at least five students and the `/about` route displays information about the application, using appropriate HTTP methods, routing, and response handling.

3. OBJECTIVE

- To install and configure the Express.js framework inside a Node.js project using npm.
- To create an HTTP server instance and make it listen on a specified port.
- To define multiple routes using the `app.get()` and `app.post()` methods.
- To return structured JSON responses using `res.json()` and HTML responses using `res.send()`.
- To implement application-level middleware for logging every incoming request.
- To handle route parameters with `req.params` and request bodies with `req.body`.
- To send appropriate HTTP status codes (200, 201, 400, 404) and to handle undefined routes.

4. THEORY

1. Express.js Framework

Express.js is a minimal and flexible web application framework built on top of the Node.js http module. It simplifies server creation, URL routing, middleware handling, and response generation, and is the most widely used backend framework in the MERN/MEAN stack.

2. Installation and Project Setup

A Node.js project is initialised with `npm init -y`, which generates `package.json`. Express is then installed as a dependency using `npm install express`, which creates the `node_modules` folder and records the dependency in `package.json`.

3. Routing

Routing determines how an application responds to a client request for a particular endpoint (a URI combined with an HTTP method). The general structure is `app.METHOD(PATH, HANDLER)`, where the handler receives the request object `req` and the response object `res`.

4. HTTP Methods

- **GET** - retrieves data from the server (used by `/`, `/students` and `/about`).

- **POST** - submits new data to the server to be created (used to add a new student).
- **PUT / PATCH** - updates an existing resource.
- **DELETE** - removes an existing resource.

5. Middleware

Middleware functions have access to `req`, `res`, and the `next()` function. They execute in the order in which they are registered with `app.use()`. `express.json()` is built-in middleware that parses incoming JSON payloads into `req.body`. A custom logger middleware is used in this program to print the method and URL of every request. Calling `next()` passes control to the next handler; failing to call it leaves the request hanging.

6. Request and Response Objects

- `req.params` - route parameters, for example the `:id` in `/students/:id`.
- `req.query` - query-string values such as `?year=2`.
- `req.body` - the parsed body of a POST or PUT request.
- `res.send()` - sends a string or HTML response.
- `res.json()` - sends a JSON response with the appropriate content type header.
- `res.status(code)` - sets the HTTP status code of the response.

7. HTTP Status Codes Used

- **200 OK** - the request succeeded.
- **201 Created** - a new resource was successfully created.
- **400 Bad Request** - required fields were missing or invalid.
- **404 Not Found** - the requested route or resource does not exist.

5. ALGORITHM / PROCEDURE

1. Start.
2. Create a project folder and initialise it with `npm init -y`.
3. Install the framework using `npm install express`.
4. Create `server.js` and import Express with `require('express')`.
5. Create the application instance using `const app = express()` and define the port number.
6. Register `express.json()` middleware to parse JSON request bodies.
7. Register a custom logger middleware that prints the HTTP method and URL of every request and calls `next()`.
8. Define an array containing the details of five students.
9. Define the route `GET /` that returns a welcome page listing the available routes.
0. Define the route `GET /students` that returns the entire student array as JSON along with a count.
1. Define the route `GET /students/:id` that searches the array using `Array.find()` and returns either the matching student or a 404 response.
2. Define the route `GET /about` that returns application information as JSON.
3. Define the route `POST /students` that validates the request body, creates a new student object, pushes it into the array, and responds with status 201.
4. Register a final middleware that returns a 404 JSON response for all undefined routes.
5. Start the server using `app.listen(PORT, callback)`.
6. Run the server with `node server.js` and verify each route in the browser or in Postman.
7. Stop.

6. PROGRAM

package.json

```
{
  "name": "express-student-server",
  "version": "1.0.0",
  "description": "Basic Express.js server with /, /students and /about routes",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "author": "K. Sirisha (A24126552260)",
  "dependencies": {
    "express": "^4.19.2"
  }
}
```

server.js

```
/**
 * server.js
 * Basic Express.js Student Server
 * Routes : / , /students , /about
 * Demonstrates server creation, routing, HTTP methods, middleware and response handling.
 */

const express = require('express'); // Import the Express framework
const app = express(); // Create an Express application instance
const PORT = 3000; // Port on which the server will listen

// ----- Built-in middleware -----
// Parses incoming JSON request bodies (required for the POST route)
app.use(express.json());

// ----- Custom application-level middleware (request logger) -----
app.use((req, res, next) => {
  const time = new Date().toLocaleTimeString();
  console.log(`[${time}] ${req.method} request received for ${req.url}`);
  next(); // Pass control to the next matching middleware / route handler
});

// ----- Sample data : list of five students -----
const students = [
  { id: 1, name: 'K. Sirisha', roll: 'A24126552260', branch: 'CSE', year: 2, cgpa: 9.1 },
  { id: 2, name: 'N. Samhavya', roll: 'A24126552099', branch: 'CSE', year: 2, cgpa: 8.8 },
  { id: 3, name: 'R. Harshitha', roll: 'A24126552104', branch: 'CSE', year: 2, cgpa: 8.5 },
  { id: 4, name: 'M. Praveen', roll: 'A24126552117', branch: 'CSE', year: 2, cgpa: 8.2 },
  { id: 5, name: 'S. Lokesh', roll: 'A24126552133', branch: 'CSE', year: 2, cgpa: 7.9 }
];

// ----- ROUTE 1 : Home route -----
app.get('/', (req, res) => {
  res.status(200).send(`
    <html>
      <head><title>Student Server</title></head>
      <body style="font-family: Arial, sans-serif; padding: 30px;">
        <h1>Welcome to the Student Management Server</h1>
        <p>This server was built using Node.js and Express.js.</p>
        <h3>Available Routes</h3>
        <ul>
          <li><a href="/students">/students</a> - returns the list of all students (JSON)</li>
          <li><a href="/about">/about</a> - information about this application</li>
        </ul>
      </body>
    </html>
  `);
});

// ----- ROUTE 2 : Students route (returns JSON) -----
app.get('/students', (req, res) => {
  res.status(200).json({
    success: true,
    count: students.length,
    data: students
  });
});

// ----- ROUTE 3 : Single student using a route parameter -----
app.get('/students/:id', (req, res) => {
  const id = parseInt(req.params.id, 10);
  const student = students.find(s => s.id === id);

  if (!student) {
    return res.status(404).json({
      success: false,
      message: `Student with id ${id} not found`
    });
  }
  res.status(200).json({ success: true, data: student });
});

// ----- ROUTE 4 : About route -----
app.get('/about', (req, res) => {
  res.status(200).json({
    application: 'Student Management Server',
    description: 'A basic REST-style server that stores and returns student details.',
  });
});
```

```

        course: 'Full Stack Web Development (23CM4121)',
        developedBy: 'K. Sirisha (A24126552260)',
        technologies: ['Node.js', 'Express.js', 'JavaScript (ES6)'],
        version: '1.0.0'
    });
});

// ----- ROUTE 5 : POST method to add a new student -----
app.post('/students', (req, res) => {
    const { name, roll, branch, year, cgpa } = req.body;

    // Validation of the incoming request body
    if (!name || !roll) {
        return res.status(400).json({
            success: false,
            message: 'Name and roll number are required fields'
        });
    }

    const newStudent = {
        id: students.length + 1,
        name, roll,
        branch: branch || 'CSE',
        year: year || 1,
        cgpa: cgpa || 0
    };

    students.push(newStudent);
    res.status(201).json({ success: true, message: 'Student added', data: newStudent });
});

// ----- 404 handler for undefined routes -----
app.use((req, res) => {
    res.status(404).json({
        success: false,
        message: `Route ${req.url} not found on this server`
    });
});

// ----- Start the server -----
app.listen(PORT, () => {
    console.log('Server is running at http://localhost:' + PORT);
    console.log('Available routes : / | /students | /students/:id | /about');
});

```

Commands to Execute

Terminal - node server.js

```

Server is running at http://localhost:3000
Available routes : / | /students | /students/:id | /about
[10:15:54 AM] GET request received for /students
[10:15:54 AM] GET request received for /about
[10:15:54 AM] GET request received for /faculty
[10:15:54 AM] GET request received for /students/3
[10:15:54 AM] POST request received for /students
[10:15:54 AM] GET request received for /
Server is running at http://localhost:3000
Available routes : / | /students | /students/:id | /about

```

7. CODE EXPLANATION

Code	Explanation
<code>require('express')</code>	Imports the Express framework that was installed through npm.
<code>const app = express()</code>	Creates an Express application instance which holds all routes and middleware.
<code>app.use(express.json())</code>	Built-in middleware that parses incoming JSON request bodies into <code>req.body</code> .
<code>app.use((req, res, next) => {...})</code>	Custom application-level middleware that logs the HTTP method and URL of every request.
<code>next()</code>	Passes control from the current middleware to the next matching handler in the stack.
<code>app.get('/', handler)</code>	Defines the home route that responds to HTTP GET requests on the root URL.
<code>app.get('/students', handler)</code>	Route that returns the array of five student records in JSON format.
<code>app.get('/students/:id', handler)</code>	Dynamic route in which <code>:id</code> is a route parameter accessible through <code>req.params.id</code> .
<code>students.find(s => s.id === id)</code>	Array higher-order method that returns the first student object matching the given <code>id</code> .
<code>app.post('/students', handler)</code>	Route that accepts a new student record through the HTTP POST method.
<code>req.body</code>	Contains the parsed JSON payload sent by the client in a POST request.
<code>res.status(200).json({...})</code>	Sets the HTTP status code and sends a JSON response with the correct content type.
<code>res.send(htmlString)</code>	Sends an HTML string as the response body for the home route.
<code>res.status(404)</code>	Returned when a requested student id or route does not exist on the server.
<code>app.listen(PORT, callback)</code>	Binds the server to port 3000 and starts listening for incoming HTTP requests.

8. TEST CASES AND EXECUTION DATA

The following test cases were recorded after starting the server and testing each route in the browser and in Postman.

Test Case	TC-01
Test / Input	Start the server with node server.js and open http://localhost:3000/ in the browser.
Expected Result	The server starts on port 3000 and the home page displays a welcome message with links to /students and /about.
Actual Result	Server started successfully, the welcome page rendered, and the logger middleware printed the GET request in the terminal.
Status	Pass

TC-01 Output:

Student Server

http://localhost:3000/

Welcome to the Student Management Server

This server was built using Node.js and Express.js.

Available Routes

- </students> - returns the list of all students (JSON)
- </about> - information about this application

200 OK

Test Case	TC-02
Test / Input	Send a GET request to http://localhost:3000/students.
Expected Result	A JSON response with status 200 containing count = 5 and the array of five student records is returned.
Actual Result	All five student records were returned correctly in JSON format with the correct count.
Status	Pass

TC-02 Output:

localhost:3000/students

http://localhost:3000/students

```
{
  "success": true,
  "count": 5,
  "data": [
    {
      "id": 1,
      "name": "K. Sirisha",
      "roll": "A24126552260",
      "branch": "CSE",
      "year": 2,
      "cgpa": 9.1
    },
    {
      "id": 2,
      "name": "N. Samhavya",
      "roll": "A24126552099",
      "branch": "CSE",
      "year": 2,
      "cgpa": 8.8
    },
    {
      "id": 3,
      "name": "R. Harshitha",
      "roll": "A24126552104",
      "branch": "CSE",
      "year": 2,
      "cgpa": 8.5
    },
    {
      "id": 4,
      "name": "M. Praveen",
      "roll": "A24126552117",
      "branch": "CSE",
      "year": 2,
      "cgpa": 8.2
    },
    {
      "id": 5,
      "name": "S. Lokesh",
      "roll": "A24126552133",
      "branch": "CSE",
      "year": 2,
      "cgpa": 7.9
    }
  ]
}
```

200 OK

Test Case	TC-03
Test / Input	Send a GET request to http://localhost:3000/about.
Expected Result	A JSON response with status 200 containing the application name, description, course, developer, and technologies is returned.
Actual Result	Application information was displayed correctly in the browser as formatted JSON.
Status	Pass

TC-03 Output:

localhost:3000/about

🔒 http://localhost:3000/about

```
{
  "application": "Student Management Server",
  "description": "A basic REST-style server that stores and returns student details.",
  "course": "Full Stack Web Development (23CM4121)",
  "developedBy": "K. Sirisha (A24126552260)",
  "technologies": [
    "Node.js",
    "Express.js",
    "JavaScript (ES6)"
  ],
  "version": "1.0.0"
}
```

200 OK

Test Case	TC-04
Test / Input	Request an undefined route: http://localhost:3000/faculty.
Expected Result	The 404 middleware returns status 404 with the message that the route was not found.
Actual Result	The 404 JSON response was returned as expected, confirming that the fallback middleware works.
Status	Pass

TC-04 Output:

localhost:3000/faculty

 http://localhost:3000/faculty

```
{
  "success": false,
  "message": "Route /faculty not found on this server"
}
```

404 Not Found

Test Case	TC-05
Test / Input	Send a POST request to /students in Postman with a valid JSON body containing name and roll.
Expected Result	Status 201 Created is returned along with the newly created student object, and the array now contains six students.
Actual Result	The new record was added successfully and returned in the response body.
Status	Pass

TC-05 Output:

Terminal - POST /students

```
$ curl -i -X POST http://localhost:3000/students \
  -H "Content-Type: application/json" \
  -d '{"name":"T. Anusha","roll":"A24126552141","branch":"CSE","year":2,"cgpa":8.6}'

HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8

{
  "success": true,
  "message": "Student added",
  "data": {
    "id": 6,
    "name": "T. Anusha",
    "roll": "A24126552141",
    "branch": "CSE",
    "year": 2,
    "cgpa": 8.6
  }
}
```

9. OUTPUT

Output 1: Express.js Student Server - Home Route in the Browser



Figure 1: Full Webpage Output of the Express.js Student Server

10. RESULT

Thus, the **Express.js Student Server** was successfully created and executed. The routes `/`, `/students`, and `/about` were implemented using appropriate HTTP methods and routing, the `/students` route returned a list of five students, and middleware, route parameters, status codes, and 404 handling were verified through the browser and Postman.